



Single machine scheduling to minimize maximum lateness subject to release dates and precedence constraints

Zhixin Liu *

Department of Management Studies, College of Business, University of Michigan - Dearborn, Dearborn, MI 48126, USA

ARTICLE INFO

Available online 18 November 2009

Keywords:

Deterministic scheduling
Maximum lateness
Release date
Precedence constraint
Branch and bound algorithm

ABSTRACT

This paper considers the problem of finding a nonpreemptive schedule for a single machine to minimize the maximum lateness with release dates and precedence constraints. A branch and bound algorithm is developed. The algorithm uses four different heuristics to find upper bounds at the initial branch node: early release date heuristic, modified Schrage's heuristic, heuristic BLOCK, and a variable neighborhood descent procedure. At each branch node, two branches evolve from a schedule found by heuristic BLOCK using a binary branching rule based on bottleneck and critical jobs, and a lower bound is obtained by optimally solving the relaxed problem with preemption. The algorithm solves 14,984 out of the 15,000 systematically generated instances with up to 1,000 jobs within 1 minute of CPU time.

© 2009 Elsevier Ltd. All rights reserved.

1. Introduction

This paper considers the problem of finding a nonpreemptive schedule for n jobs on a single machine, to minimize the maximum lateness subject to given release dates and precedence constraints. Specifically, suppose that a set J of n jobs $1, \dots, n$ is to be processed nonpreemptively on a single machine, which can execute at most one job at a time. Each job j becomes available at time r_j , and requires a processing time p_j . Moreover, a precedence relation $\rightarrow$ between the jobs is given: if $j \rightarrow k$, then it is required that job j is completed before job k can start. Associated with each job j is a due date d_j which specifies before what time job j 's processing is preferred to be completed. If job j has completion time C_j and due date d_j , then a lateness cost of $L_j = C_j - d_j$ is incurred for job j . The objective is to find a feasible schedule such that the maximum lateness cost, $L_{\max} = \max_j \{C_j - d_j\}$, is minimized.

Following the standard $\alpha|\beta|\gamma$ classification scheme for scheduling problems [7], α indicates the scheduling environment, β describes the job characteristics or restrictive requirements, and γ specifies the objective function to be minimized. The scheduling problem studied in this work is denoted by $1|prec, r_j|L_{\max}$.

Many applications in practical problems are captured by problem $1|prec, r_j|L_{\max}$. Release dates describe the available time of jobs, due dates measure the preferable time for jobs to be completed, and precedence constraints specify partial orders for the jobs to be scheduled. Also, this problem has another description: to minimize the makespan with job release dates,

precedence constraints, and delivery times. For each job j , delivery time q_j is defined as the time required for the job to be delivered when its processing has been completed. Any set of processed jobs can be delivered simultaneously. The makespan of a schedule is the earliest time to complete delivery of all the jobs. The reason for this equivalence is that each delivery time q_j can be replaced by a due date $d_j = K - q_j$, where K is a constant, and then an equilibrant instance of problem $1|prec, r_j|L_{\max}$ is created.

Moreover, problem $1|prec, r_j|L_{\max}$ consists of several subproblems for many other more complicated scheduling problems and can be used to obtain useful bounds or optimal schedules directly. First, it is closely related to job shop scheduling [1,3]. In a job shop, the sequence of machines for each job is prescribed, and each machine can process at most one job at a time. For each job, if we consider its processing on a machine as an operation, then these operations have implied release dates and precedence relations. Second, as shown by Schutten et al. [18], problem $1|r_j, agr-prec|L_{\max}$ defines a lower bound for a single machine scheduling problem with release dates, due dates and family setup times to minimize the maximum lateness, where *agr-prec*, the agreeable precedence, means that a precedence constraint $j \rightarrow k$ indicates $d_j \leq d_k - p_k$. The idea behind this transformation is that setup times can be modeled as jobs with specific processing times, release dates, due dates, and precedence relations. Third, problem $1|prec, r_j|L_{\max}$ is useful to model disruptions to existing schedules when rescheduling is necessary [8]. Specifically, when a job needs to be rescheduled, changes to its original starting time, completion time, and sequence position should be limited to a given extent or measured as a cost, which can be modeled by release date, due date and precedence constraint.

Due to these extensive applications, it is important to develop efficient algorithms to solve problem $1|prec, r_j|L_{\max}$. The problem

* Tel.: +1 313 593 8386; fax: +1 313 271 9836.
E-mail address: zhixin@umd.umich.edu

is unary *NP*-hard, even without precedence constraints [6,15]. In specific, three special cases of this problem are discussed as follows.

First, if all release dates are equal, the problem is solvable in $O(n^2)$ time (see [14]). Lawler's method was extended by Baker et al. [2] to solve the problem $1|pmtn, prec, r_j|f_{\max}$, where a monotone nondecreasing cost function f_j is associated with each job j to define a cost of $f_j(C_j)$ when job j completes processing at time C_j . The term *pmtn* means that an arbitrary number of preemptions is allowed for each job without punishment, where a preemption is incurred when the processing of a job is interrupted to stop and restart later to process the remaining part.

Second, the well studied problem $1|r_j|L_{\max}$ is a special case of problem $1|prec, r_j|L_{\max}$ when no precedence constraints present. Even though problem $1|r_j|L_{\max}$ is also unary *NP*-hard [15], it is a relatively easy problem to solve heuristically [10]. McMahon and Florian [16] and Carlier [5] both provide efficient branch and bound algorithms to solve this problem. The latter work is similar to the previous one, but utilizes a different branching technique and solves instances with up to thousands of jobs quickly, rarely requiring the number of branch nodes to be greater than $2n$.

Third, problem $1|agr-prec, r_j|L_{\max}$ with agreeable precedence structure deserves discussion. Schutten et al. [18] introduce this problem and provide a branch and bound algorithm to solve randomly generated instances with up to 50 jobs, among which almost all the instances with up to 40 jobs are optimally solved. They also show that problem $1|agr-prec, r_j|L_{\max}$ defines a lower bound for a single machine scheduling problem to minimize the maximum lateness, with release dates, due dates and family setup times.

In this paper, an efficient branch and bound algorithm is developed to solve the problem $1|prec, r_j|L_{\max}$. We use Baker et al. [2]'s method to solve the relaxed problem with preemption to obtain lower bounds, and develop and apply four heuristics to find upper bounds. In the branch and bound procedure, an optimality condition is applied to eliminate branch nodes of the search tree. In the computational study, our experimental design follows the schemes proposed by Hall and Posner [9].

This paper is organized as follows. Section 2 provides several properties to simplify the problem, to recognize optimality of schedules, and to develop a binary branching rule. Section 3 describes a branch and bound algorithm in detail, including a lower bound technique and four heuristics to obtain upper bounds. In Section 4, extensive computational studies of the branch and bound algorithm are provided. Finally, Section 5 contains a summary of this paper.

2. Properties

In problem $1|prec, r_j|L_{\max}$, combined with the original release dates, the processing times and precedence constraints can be applied to redefine tighter release dates. The following modification procedure is a general method to define new release dates, see, for example, Lageweg et al. [13].

Release dates modification (RDM). Step 1: Renumber the jobs in topological order such that for any two jobs j and k , $j < k$ when $j \rightarrow k$.

Step 2: Set $r_k = \max\{r_k, \max_{j \rightarrow k}\{r_j + p_j | j \rightarrow k\}\}$, for k from 2 to n .

Henceforth, when a release date r_j for each job j is mentioned, it is a modified release date. After this modification, for any two jobs j and k , we have that if $j \rightarrow k$, then $r_j < r_k$. Recall that a *semiactive schedule* is a schedule in which no job can be moved earlier without changing the sequence or violating a constraint [4]. Obviously, to find an optimal schedule for problem $1|prec, r_j|L_{\max}$,

it is sufficient to only consider semiactive schedules. The following theorem is useful to reduce instance size of problem $1|prec, r_j|L_{\max}$.

Theorem 1. For a given instance of problem $1|prec, r_j|L_{\max}$, suppose LB is a lower bound of the optimal value of maximum lateness. In a feasible schedule σ , suppose that job j has $r_j = C_j - p_j$. If the jobs preceding job j all have lateness no greater than LB , and the jobs succeeding job j all have release dates no earlier than job j , then to find an optimal schedule, it is sufficient to consider only job j and the jobs succeeding it.

Proof. There is no need to further consider the jobs preceding job j , since that the maximum lateness of these jobs is no greater than LB and that the jobs succeeding job j cannot interrupt their schedule. $\square$

Next, we introduce the definition of *block*, which is used in our lower and upper bound techniques.

Definition 1. In a feasible schedule σ of problem $1|prec, r_j|L_{\max}$, a block is a set of jobs $j_1, \dots, j_k$ successively processed in that sequence, $1 \leq k \leq n$, where $r_{j_1} = C_{j_1} - p_{j_1}$, $r_{j_i} < C_{j_i} - p_{j_i}$ for $i = 2, \dots, k$, and $r_{j_{k+1}} = C_{j_{k+1}} - p_{j_{k+1}}$, where job j_{k+1} is the job processed immediately after job j_k ; or job j_k is the last job in schedule σ .

By definition, jobs within a block are processed without idle times between them. Note that a block is not a maximum set of jobs processed without inserted idle time. If the release date of a job exactly corresponds to the completion time of the job that immediately precedes it, then the two jobs belong to different blocks. Such a definition is necessary to guarantee the correctness of algorithm PMTN, which we introduce in Section 3.1. Let B_i denote the set of jobs in a block i , and S_{B_i} and C_{B_i} denote the starting time of the first job in B_i and the completion time of the last job in B_i , respectively.

In a schedule σ for problem $1|prec, r_j|L_{\max}$, a *bottleneck job* is a job that has lateness equal to $L_{\max}(\sigma)$. Next we define *critical block*.

Definition 2. In a feasible schedule of problem $1|prec, r_j|L_{\max}$, a critical block is a set of jobs $j_1, \dots, j_k$ successively processed in that sequence, $1 \leq k \leq n$, where job j_k is a bottleneck job, no idle time exists among jobs $j_1, \dots, j_k$, and job j_1 starts processing at its release date, and has the minimum release date among jobs $j_1, \dots, j_k$.

Note that a semiactive feasible schedule of problem $1|prec, r_j|L_{\max}$ is naturally formed by blocks and contains at least one bottleneck job, but may not contain any critical block. Consider an instance with two jobs, where $r_1 < r_2$. If job 1 is processed after job 2 and job 2 is not a bottleneck job, then no critical block exists in the schedule. Next, we prove a theorem that is useful to recognize optimality of schedules containing critical blocks.

Theorem 2. In a feasible schedule σ of problem $1|prec, r_j|L_{\max}$, suppose that jobs $j_1, \dots, j_k$ processed in that sequence form a critical block. Schedule σ is optimal if for each $i = 1, \dots, k$, job j_i either has due date no less than d_{j_k} , or is an enforced predecessor for at least one job within jobs $j_{i+1}, \dots, j_k$ by precedence constraints.

Proof. First, note that there are no idle times among jobs $j_1, \dots, j_k$ in schedule σ from Definition 2. Further, by Definition 2, $C_{j_1} - p_{j_1} = r_{j_1}$. Hence, since that job j_1 has the earliest release date in jobs $j_1, \dots, j_k$, and job j_k has the largest due date within jobs $j_1, \dots, j_k$ that are not required to precede other jobs within jobs $j_1, \dots, j_k$, we have that in any feasible schedule, there exist a job with completion time no earlier than C_{j_k} and due date no larger

than d_{j_k} . Therefore, no feasible schedule can have maximum lateness less than $L_{j_k}(\sigma)$. $\square$

For a feasible schedule of a problem instance with n jobs, the optimality condition in Theorem 2 can be tested in $O(n^2)$ time through a straightforward enumeration. If a critical block exists but the optimality condition in Theorem 2 is not satisfied, then there exists a job j_i among jobs $j_1, \dots, j_{k-1}$ that has $d_{j_i} > d_{j_k}$ and is not enforced to precede any jobs within jobs $j_{i+1}, \dots, j_k$ by precedence constraints, which we call a *critical job*.

Suppose that job j_i is the latest processed critical job within a critical block of jobs $j_1, \dots, j_k$. It is easy to see, in an optimal schedule, job j_i is processed either before or after the bottleneck job j_k . This observation provides us an idea to branch the current schedule to two branch nodes: one node requires job j_i to precede job j_k , and the other node requires job j_i to succeed job j_k . Our branch and bound algorithm is based on this *binary branching rule*. Note that this branching method can be strengthened if there is no precedence constants. In this case, job j_i has a due date larger than any job within $j_{i+1}, \dots, j_k$, and it is sufficient to consider two branch nodes where job j_i is processed either before or after all jobs $j_{i+1}, \dots, j_k$ [5].

3. Branch and bound algorithm

In order for the binary branching rule to work efficiently, we need to find methods that recognize tight lower bound, and to find efficient heuristics that obtain schedules having critical blocks. In this section, lower bound methods are introduced in Section 3.1, upper bound heuristics are developed in Section 3.2, and the overall branch and bound algorithm is designed in Section 3.3.

3.1. Lower bounds

Assuming that in a feasible schedule for problem $1|prec, r_j|L_{\max}$, there exists a critical block containing jobs $j_1, \dots, j_k$ processed in that sequence. Let $d_{\max}$ be the maximum due date of jobs within $j_1, \dots, j_k$ that are not enforced to precede any other jobs within $j_1, \dots, j_k$ by precedence constraints. Obviously, $C_{j_k} - d_{\max}$ is a lower bound of the maximum lateness of any feasible schedule. We name this lower bound as *critical block lower bound*.

The optimal value of the maximum lateness for the relaxed problem $1|pmtn, prec, r_j|L_{\max}$ is also a lower bound, which we name as *preemptive lower bound*. Horn [12] and Baker et al. [2] develop different algorithms to optimally solve the relaxed problem with the same time complexity $O(n^2)$. In this paper, the method developed by Baker et al. is used and is denoted by PMTN. To achieve completeness of this paper, algorithm PMTN is briefly described as follows.

PMTN. *Step 1: Initialization:* Schedule the jobs in nondecreasing order of r_j each as early as possible, and renumber the jobs in this order. Divide the schedule into blocks, and mark each of them as unadjusted.

Step 2: Adjustment.

Step 2.1: If no unadjusted block exists, then output the schedule and stop; otherwise, find an unadjusted block i .

Step 2.2: From all the jobs in B_i with no successors in B_i defined by precedence constraints, find a job j with the maximum due date.

Step 2.3: Schedule all of the jobs in $B_i \setminus \{j\}$ nonpreemptively in nondecreasing r_j order, starting each as early as possible, and schedule job j with completion $C_j = C_{B_i}$, preempting if necessary. If

idle time between jobs in $B_i \setminus \{j\}$ results, then $B_i \setminus \{j\}$ is partitioned into several blocks, and we record all the new blocks and mark them as unadjusted, return to Step 2.1; otherwise set $B_i = B_i \setminus \{j\}$. If $B_i \neq \emptyset$, then mark block i as unadjusted and return to Step 2.2, else return to Step 2.1.

Step 1 of PMTN finds a feasible schedule since that $j \rightarrow k$ indicates $r_j < r_k$ from procedure RDM in Section 2. In a schedule formed at Step 1, blocks are *independent*, due to the fact that for any job j in a block i , it is infeasible to start earlier than S_{B_i} , and to find an optimal preemptive schedule, it is unnecessary to complete later than C_{B_i} . Independent blocks are individually considered in PMTN. Note that PMTN may find a schedule without preemption, and in this case, an optimal schedule for problem $1|prec, r_j|L_{\max}$ is found.

3.2. Upper bounds

The following four heuristics are used to find upper bounds for the branch and bound algorithm at the initial branch node. Foremost, a method is introduced that can be implemented to reduce instance size, which proceeds Theorem 1.

Earliest release dates (ERD). *Step 0:* Suppose LB is a lower bound for the optimal value of $L_{\max}$ found by PMTN.

Step 1: Schedule the jobs in nondecreasing order of r_j each as early as possible, where ties are broken by choosing the job with earlier due date first.

Step 2: Find the maximum lateness $L_{\max}^k$ of the first k jobs for $k = 1, \dots, n$, successively. For k from n to 2, if $r_k = C_k - p_k$ and $L_{\max}^{k-1} \leq LB$, then eliminate the first $k-1$ jobs and stop.

Heuristic ERD uses the schedule defined by nondecreasing order of r_j , and then reduces instance size, with time complexity of $O(n \log n)$. The schedule resulted in heuristic ERD is feasible since $j \rightarrow k$ indicates that $r_j < r_k$ from procedure RDM in Section 2.

Next, Schrage's heuristic [17], which is optimal for problem $1|r_j, p_j = 1|L_{\max}$, is modified to solve the more general problem $1|prec, r_j|L_{\max}$.

Schrage's heuristic (Modified). *Step 1:* Let $J_a = \emptyset$ be the set of jobs that has been scheduled and let $J_b = J$ be the set of all other jobs.

Step 2: Compute the completion time C_a of the last completed job in J_a , $C_a = 0$ if $J_a = \emptyset$, and the minimum release date r_b of the jobs in J_b . Set $t = \max\{C_a, r_b\}$.

Step 3: For all the jobs in J_b with release dates no greater than t and having no jobs in J_b preceding it due to precedence constraints, find a job j with the minimum due date, and schedule the selected job at the end of the partial schedule containing J_a . Set $J_a = J_a \cup \{j\}$ and $J_b = J_b \setminus \{j\}$. If $J_b = \emptyset$, output the schedule and stop, else return to Step 2.

Again, Schrage's heuristic is implementable due to the fact that $j \rightarrow k$ indicates $r_j < r_k$. With the jobs numbered in topological order, Schrage's heuristic can be implemented in $O(n^2)$ time.

Next, heuristic BLOCK is developed, which follows the idea of PMTN, but can find a nonpreemptive schedule for problem $1|prec, r_j|L_{\max}$.

Heuristic BLOCK. *Step 1: Initialization:* Schedule the jobs in nondecreasing order of r_j each as early as possible, renumber the jobs in this order, and divide the schedule into blocks. For each block i , mark it as unadjusted and compute its maximum lateness $L_{\max}^i$.

Step 2: Adjustment.

Step 2.1: If no unadjusted block exists, then output the schedule and stop; otherwise, from all the unadjusted blocks, find a block i with the largest maximum lateness. If $L_{\max}^i$ is no greater than the maximum lateness of jobs within all the adjusted blocks, then output the schedule and its maximum lateness, and stop.

Step 2.2: Set $\bar{B}_i = B_i$.

Step 2.3: Set $\hat{B}_i = \bar{B}_i$.

Step 2.4: From all the jobs in $\hat{B}_i$ with no successors in $\bar{B}_i$ defined by precedence constraints, find a job j with the maximum due date.

Step 2.5: Schedule all the jobs in $\bar{B}_i \setminus \{j\}$ in nondecreasing r_j order each as early as possible. If idle time between time S_{B_i} and $C_{\bar{B}_i} - p_j$ results, then let $\hat{B}_i = \hat{B}_i \setminus \{j\}$, and return to Step 2.4; otherwise, schedule job j to be completed at time $C_{\bar{B}_i}$ and let $\bar{B}_i = \bar{B}_i \setminus \{j\}$. If $\bar{B}_i \neq \emptyset$, then return to Step 2.3, else mark block i as adjusted and return to Step 2.1.

To derive the time complexity of heuristic BLOCK, note that there are at most n values of $C_{\bar{B}_i}$ to consider, and $O(n^2)$ time is required to determine a job to be completed at each $C_{\bar{B}_i}$. Therefore, heuristic BLOCK has a time complexity of $O(n^3)$. However, computational studies in this research suggest that for most instances only $O(n)$ computation time is needed to determine a job to be completed at each $C_{\bar{B}_i}$. Hence, the average computing time of heuristic BLOCK is approximately $O(n^2)$, which is the same as of PMTN.

The fourth heuristic is a variable neighborhood descent (VND) procedure. We recommend Hansen and Mladenović [11] as a useful introduction for the VND to solve different problems. Our VND procedure uses six types of neighborhood. We use F_j and A_j to denote the set of jobs that are processed before and after job j , respectively, for $j \in J$. In the below description of neighborhood moves, it is supposed that job j is the earliest processed bottleneck job in the current schedule. In addition, if there is machine idle time before the processing of job j , suppose that job h is immediately preceded by machine idle time and there is no machine idle time between the processing of jobs h and j . To reduce the maximum lateness, job j must be completed earlier, and therefore the set of jobs processed between job h (inclusive) and job j (noninclusive) must be changed. Let $N_l(\sigma)$ be the type l neighborhood for a schedule σ , and N_l for short if there is no ambiguity, for $l = 1, \dots, 6$. Neighborhood structures which achieve this change are defined as follows:

N_1 : remove job j and insert it immediately before a job i , where $i \in F_j$;

N_2 : remove a job i and insert it immediately after job j , where $i \in F_j \setminus F_h$;

N_3 : swap a job i and job j , where $i \in F_j$;

N_4 : if h exists, remove job h and insert it immediately after a job i , where $i \in A_h$;

N_5 : if h exists, remove a job i and insert it immediately before job h , where $i \in A_h \setminus (A_j \cup \{j\})$;

N_6 : if h exists, swap job h and a job i , where $i \in A_h$.

Note that for any neighbor, feasibility needs to be maintained. Specifically, if a precedence constraint is violated in the resulting schedule, then this neighbor is not considered. For a neighbor, the corresponding schedule is computed by scheduling each job in the sequence as early as possible, subject to the release date constraints.

A neighbor resulting from one of these moves is *preferred* to the previous schedule if the maximum lateness is smaller. In addition, we have clarifications concerning ties. First consider N_1 , N_2 and N_3 . In the case of equal maximum lateness values, the

schedule with the earlier completion time of the job in the position previously occupied by job j is preferred. If there is still a tie, then the neighbor is preferred to the previous schedule if $d_j < d_i$ or if $d_j = d_i$ and $j < i$. Second, consider N_4 , N_5 and N_6 . In the case of equal maximum lateness values, the schedule with the earlier completion time of the job in the position previously occupied by job i is preferred. If there is still a tie, then the neighbor is preferred to the previous schedule if $r_i < r_h$ or if $r_i = r_h$ and $i < h$. Now we describe the VND procedure as below.

Variable Neighborhood Descent (VND). *Initialization:* Apply Schrage's heuristic and heuristic Block to find two feasible schedules, choose the found schedule with smaller maximum lateness, and denote it by σ .

Search neighborhoods: Conduct the following operations for $l = 1 - 6$.

Execute the following until none of $N_l(\sigma)$ is preferred to σ , for the current schedule σ .

Find a preferred neighbor σ' of σ with the smallest maximum lateness, and set $\sigma = \sigma'$.

Termination test: If $L_{\max}(\sigma)$ has been reduced during the step of Search Neighborhoods, then return to the Search Neighborhoods step; otherwise, stop.

3.3. Branch and bound

Our branch and bound algorithm uses the binary branching strategy introduced in Section 2. The search tree evolves by branching to two nodes from a currently active branch node, where a critical job is enforced to be processed before and after the bottleneck job in the same critical block, respectively. Wherever possible, these branch nodes are eliminated using the optimality condition in Theorem 2 or a lower bound. Specifically, for a branch node, when PMTN achieves a schedule with the maximum lateness no less than the current upper bound, or with no preemptions, the branch node is eliminated.

Procedure ERD, Schrage's heuristic, and heuristic BLOCK all find schedules with critical blocks, and thus can be used as heuristics at each branching node. However, VND may not find a schedule with any critical block, and thus is only used at the root node.

We first consider a simple branch and bound algorithm based on Schrage's heuristic (SBB) that only uses the critical block lower bound and an upper bound found by Schrage's heuristic. Algorithm SBB serves as a heuristic if we limit the number of branching nodes that can be evolved, and also as a benchmark to evaluate the efficiency of our next complex branch and bound algorithm. Note that algorithm SBB follows the idea of Carlier [5]'s method solving problem $1|r_j|L_{\max}$.

An overview of the solution procedure is now provided, which uses branch and bound if matching lower and upper bounds are not found at the root node of the search tree.

Solution procedure overview. *Step 1:* Use Step 1 of ERD to compute the first upper bound. If the schedule obtained satisfies the optimality condition in Theorem 2, an optimal solution has been found by ERD, and the procedure terminates.

Step 2: Find a lower bound using PMTN. If no job is preempted, then an optimal solution has been found and the procedure terminates. Else, if the lower bound equals the upper bound found in Step 1, an optimal solution has been found by ERD, and the procedure terminates. Otherwise, apply Step 2 of ERD for the schedule found by Step 1, to reduce instance size.

Step 3: Compute an upper bound using Schrage's heuristic. If the lower and upper bounds are equal, or the optimality condition in

Theorem 2 is satisfied, then an optimal solution has been found by Schrage's heuristic, and the procedure terminates.

Step 4: Use BLOCK to compute a third upper bound. If the procedure results in a schedule with the same maximum lateness as the lower bound, or the schedule meets the optimality condition in Theorem 2, an optimal solution has been found by BLOCK, and the procedure terminates.

Step 5: Apply VND to compute a fourth upper bound. If VND yields a solution with the same maximum lateness as the lower bound, or the optimality condition in Theorem 2 is satisfied, then an optimal solution has been found by VND, and the procedure terminates. Otherwise, find the best of the upper bounds computed in Steps 1, 3, 4 and 5.

Step 6: Apply algorithm SBB with up to 100 branch nodes (SBB100) to compute a fifth upper bound. If SBB100 finds an optimal schedule, the procedure terminates.

Step 7: Apply the binary branch rule to search for an optimal schedule, with PMTN, BLOCK, and the optimality condition in Theorem 2 applied at each branch node. A computation time limit of 1 CPU minute is imposed, at which stage the branch and bound algorithm terminates even if an optimal solution has not been found.

4. Computational studies

The branch and bound algorithm is tested on 15,000 problem instances, which are randomly generated following the data generation schemes established by Hall and Posner [9]. In their paper, Hall and Posner propose specific principles for the generation of four most widely used types of data in scheduling problems: release dates, finish dates, precedence constraints, and machine routing and speeds, among which the first three are used in problem 1|*prec*, $r_j|L_{\max}$. Problem instances generated following these principles are shown to have desirable properties for computational evaluation: *variety*, *practical relevance*, *scale invariance*, *size invariance*, *regularity*, *describability*, *efficiency* and *parsimony*; see Hall and Posner [9] for a detailed description.

We first select $n \in \{20, 40, 60, 80, 100, 200, 400, 600, 800, 1000\}$ and generate $p_j \sim UI[1, 50]$. Hall and Posner suggest to generate release dates in ways such that $r_1 = 0$ and $r_j = g_j(p_j, r_{j-1}) + X_j$, $j = 2, \dots, n$, where $g_j(p_j, r_{j-1})$ is a bivariate function of the processing time and the prior release date, and X_j is a random variable generated from a probability distribution. We use $r_1 = 0$ and $r_j = r_{j-1} + R_j$, where $R_j \sim UI[1, 25]$, $UI[1, 50]$ or $UI[1, 75]$, for $j = 2, \dots, n$.

For job due dates, the generation method proposed by Hall and Posner [9] uses $d_j = h_j(p_1, \dots, p_n) + X_j$, $j = 1, \dots, n$, where X_j is a random variable generated from a probability distribution. Schutten et al. [18] define the interval $[r_j + p_j, r_j + p_j + k\bar{p}]$ from which d_j is uniformly distributed, where k is a constant integer and $\bar{p}$ is the mean of job processing time. Combining these two data generation procedures, we use $d_j = r_j + p_j + \delta_j$, where $\delta_j \sim UI[1, 50n]$, $UI[1, 50n/2]$, $UI[1, 50n/5]$, $UI[1, 50n/10]$ or $UI[1, 50n/20]$. Note that in our experimental design, it is likely to have release dates and due dates compatible with each other, which means that for any two jobs j and k , $r_j < r_k$ indicates $d_j < d_k$. This compatibility represents practical problems.

For generating precedence constraints, a traditional technique is to first specify a parameter D , where $0 < D < 1$, and then for each pair of jobs i and j , where $1 \leq i < j \leq n$, a random number γ_{ij} is generated from the uniform distribution over the interval $[0, 1]$. If $\gamma_{ij} < D$, a precedence constraint $i \rightarrow j$ is included. The requirement that $i < j$ ensures the existence of feasible schedule. However, due

to the transitivity of the precedence constraints, i.e. $i \rightarrow j$ and $j \rightarrow k$ implies $i \rightarrow k$, the above procedure has difficulty to control the number, or the density of the precedence constraints. Let $P_{ij} = \Pr\{i \rightarrow j \text{ exists in the immediate precedence constraints}\}$, and let $D = \text{target density of the precedence constraints} = \Pr\{i \rightarrow j \text{ exists in the precedence constraints}\}$, for $1 \leq i < j \leq n$. Hall and Posner [9] provide the following equation to calculate P_{ij} ,

$$P_{ij} = \frac{D(1-D)^{j-i-1}}{1-D(1-(1-D)^{j-i-1})}. \quad (1)$$

Eq. (1) is used in this paper to determine the probability of generating a precedence constraint $i \rightarrow j$ to achieve the desired density D , where $D \sim 0.0, 0.1, \dots, 0.9$. The case with $D = 0.0$ is included for the sake of completeness, which means that there is no precedence constraints. Without precedence constraints, problem 1| $r_j|L_{\max}$ can be solved more efficiently since the binary branching rule can be more effectively applied, as discussed in Section 2. The precedence constraints are likely but not guaranteed to be compatible with due date, but the RDM procedure defined in Section 2 guarantees that for any two jobs j and k , $j \rightarrow k$ implies $r_j < r_k$.

For each value of n and for each of the 150 combinations of the other parameters, 10 problem instances are randomly generated. The programs for all the algorithms are coded in the C++ language, compiled by the Microsoft Visual C++ compiler, and run on a Pentium 4 desktop computer with 3 GHz speed and 2 GB memory.

We first try to solve problem instances using algorithm SBB. Many instances with $n = 60$ cannot be solved optimally within 1 CPU minute. Computational results for instances with 20 and 40 jobs are summarized in Table 1, where UNS denotes the number of unsolved instances within 1 CPU minute, AT and MT denote the mean and maximum computation times of the solved instances in CPU seconds, respectively, and NODE denotes the mean number of branch nodes evolved by algorithm SBB excluding the root node for the solved instances. The computational burden of algorithm SBB increases fast with the increment of number of jobs, and only small size instances can be solved optimally in a reasonable amount of time.

For our complex branch and bound algorithm, Tables 2–5 summarize the computational results for all the 15,000 instances. In all these tables, ERD, PMTN, Schrage, BLK, VND, SBB100 and BB

Table 1
Performance of algorithm SBB.

n	UNS	AT	MT	NODE
20	0	< 0.01	0.02	7
40	3	0.16	56.02	7116

Table 2
Effect of n on performance of algorithms.

n	ERD	PMTN	Schrage	BLK	VND	SBB100	BB	UNS	AT	MT
20	601	200	181	137	272	106	3	0	< 0.01	0.02
40	455	165	210	259	309	83	18	1	< 0.01	0.17
60	355	154	192	368	316	95	20	0	< 0.01	0.27
80	315	134	166	431	339	76	39	0	< 0.01	0.14
100	294	132	149	479	336	65	44	1	< 0.01	0.91
200	246	97	101	562	388	47	58	1	0.01	1.95
400	196	76	114	591	428	31	64	0	0.03	0.78
600	163	84	108	589	451	22	80	3	0.12	45.61
800	167	65	108	588	462	23	82	5	0.19	16.70
1000	167	57	96	614	461	26	76	3	0.38	33.91

Table 3
Effect of release date on performance of algorithms.

	RD	ERD	PMTN	Schrage	BLK	VND	SBB100	BB	UNS	AT	MT
R_1	296	966	438	2685	423	57	133	2	0.11	8.75	
R_2	599	152	398	1633	1594	270	342	12	0.10	45.61	
R_3	2064	46	589	300	1745	247	9	0	<0.01	1.30	

Table 4
Effect of due date on performance of algorithms.

	DD	ERD	PMTN	Schrage	BLK	VND	SBB100	BB	UNS	AT	MT
δ_1	706	136	115	641	1195	72	134	1	0.08	5.25	
δ_2	615	196	134	809	1013	104	124	5	0.07	8.75	
δ_3	513	224	289	1043	712	109	107	3	0.06	3.55	
δ_4	537	288	355	1078	508	146	84	4	0.10	45.61	
δ_5	588	320	532	1047	334	143	35	1	0.05	3.39	

Table 5
Effect of precedence density on performance of algorithms.

D	ERD	PMTN	Schrage	BLK	VND	SBB100	BB	UNS	AT	MT
0.0	112	6	454	41	661	167	56	3	0.09	6.86
0.1	121	17	133	487	551	92	96	3	0.10	16.70
0.2	148	25	110	578	498	64	77	0	0.12	45.61
0.3	165	56	119	572	452	59	74	3	0.09	4.20
0.4	231	87	110	562	395	58	54	3	0.09	33.91
0.5	276	102	133	555	342	42	50	0	0.06	5.25
0.6	314	148	123	552	296	32	35	0	0.05	3.53
0.7	389	184	109	507	256	30	24	1	0.05	2.98
0.8	531	236	73	444	190	16	9	1	0.04	2.78
0.9	672	303	61	320	121	14	9	0	0.03	1.38

denote the number of instances with optimal solutions found by ERD, PMTN, Schrage's heuristic, BLOCK, VND, SBB100, and branch and bound evolving more than the root node, respectively; UNS denotes the number of unsolved instances within 1 CPU minute; AT and MT denote the mean and maximum computation time of the solved instances in CPU seconds, respectively. Detailed discussions for each table are as follows.

Table 2 summarizes the effect of instance size on the performance of our algorithms. As the number of jobs increases, ERD, PMTN, Schrage's heuristic and SBB100 become less powerful at finding optimal schedules, but BLOCK and VND solve more instances optimally. The branch and bound is powerful since it solves almost all the instances left unsolved. The mean computing time increases with instance size, and the worst case computing time roughly follows the same trend with a few exceptions. Also, with the increment of n , the number of unsolved instances increases. Within 1 minute of CPU time, our algorithm solves 14,986 out of 15,000 instances.

Table 3 summarizes the effect of distribution of job release dates on the performance of our solution methods, where column RD denotes the three kinds of release dates considered, R_1 , R_2 and R_3 , respectively. The most notable phenomena is that instances with jobs having large difference on their release dates are relatively easy to solve. Intuitively, large difference of release dates reduces the number of semiactive feasible schedules, and thus an optimal schedule is easy to find.

Table 4 summarizes the effect of distribution of job due dates on the performance of our algorithms, where column DD denotes the five kinds of due dates generated. Overall there is not much

Table 6
Detailed results for the branch and bound algorithm.

n	n BSize	BB	NODE	AT	MT
20	19	3	12	<0.01	<0.01
40	36	18	217	0.02	0.17
60	54	20	221	0.03	0.27
80	64	39	34	0.02	0.14
100	80	44	62	0.04	0.91
200	154	58	70	0.13	1.95
400	273	64	5	0.28	0.78
600	446	80	22	1.41	45.61
800	579	82	10	1.68	16.70
1000	685	76	7	3.19	33.91

difference in computing time between different δ_i values, for $i = 1, \dots, 5$. Note that by definition the value of $L_{\max}$ can be either positive or negative, so even when all the jobs can be scheduled on time, it is still challenging to minimize the maximum lateness. However, for different heuristics, δ_i has different effects. ERD and VND perform worse with the range increase of δ_i , while PMTN, Schrage's heuristic, BLOCK and SBB100 become more useful with the same trend of δ_i .

Table 5 summarizes the effect of precedence density on the performance of the solution approach, where column D denotes the 10 types of precedence densities considered. A useful observation is that, on average, instances with higher precedence density require less time to solve, especially when $D \geq 0.5$. One possible reason is that higher precedence density reduces the number of semiactive feasible schedules and thus makes an optimal schedule easy to find.

Table 6 provides more detailed information about the operation of our branch and bound algorithm. For instances solved by the branch and bound with more than the root node, n BSize denotes the mean number of jobs within the Largest block found by ERD, BB denotes the number of instances, NODE denotes the mean number of branch nodes evolved by the branch and bound algorithm excluding the root node, and AT and MT denote the mean and maximum computation times in CPU seconds, respectively. Besides the instance size n , the maximum block size is an indicator of the computational burden incurred by the branch and bound algorithm, since that at each branch node, both PMTN and BLOCK treat each block individually. The main observation here is that for those solved instances, the mean number of evolved branch nodes does not monotonously increase with instance size, and typically, instances are optimally solved within 300 nodes. Our computational results further show that among the 14 instances unsolved within 1 CPU minute, five instances are solved with 228 CPU seconds and 9098 branch nodes on average, and the other nine instances are unsolved even when an average of 1.4 million branch nodes are enumerated. Essentially, this is due to the fact that problem $1|prec, r_j|L_{\max}$ is unary NP-hard.

Besides the careful design of our solution approach, there are two reasons for problem $1|prec, r_j|L_{\max}$ to be solved thus efficiently. First, the objective $L_{\max}$ is a bottleneck measure, and thus even for large number of jobs, it is possible that only the schedules of a few jobs affect the maximum lateness. Second, release dates and precedence constraints may reduce the number of semiactive feasible schedules.

Finally, in case that the efficiency of our algorithms is due to the high degree of compatibility of release dates, due dates, and precedence constraints, totally randomized instances are also tested. Again, $n \in \{20, 40, 60, 80, 100, 200, 400, 600, 800, 1000\}$ is selected and $p_j \sim U[1, 50]$ is generated. Release dates and due dates are all integers evenly distributed in $U[1, 25n]$, for each

Table 7

Performance of algorithms for totally randomized instances.

<i>n</i>	ERD	PMTN	Schrage	BLK	VND	SBB100	BB	UNS	AT	MT
20	18	20	4	20	25	13	0	0	< 0.01	< 0.01
40	7	17	2	35	32	4	3	0	< 0.01	0.02
60	4	16	2	39	33	2	4	0	< 0.01	0.02
80	4	5	1	47	35	0	8	0	< 0.01	0.03
100	1	11	0	52	31	1	4	0	< 0.01	0.03
200	0	3	1	59	27	0	10	0	0.01	0.08
400	0	2	0	58	26	0	14	0	0.06	0.47
600	0	0	0	52	29	0	19	2	0.18	1.09
800	0	0	0	49	31	0	20	0	0.46	5.92
1000	0	0	1	56	22	0	19	2	0.70	4.22

value of n . The precedence constraints are generated the same as for the 15,000 systematically generated instances. For each parameter combination, 10 instance are generated, and altogether $10 \times 10 \times 10 = 1000$ instances are investigated. The computational results are summarized in Table 7. Only four instances are not solved within 1 CPU minute. On average, totally randomized instances are not more difficult to solve by our algorithms.

5. Summary

This paper considers the problem of finding a nonpreemptive schedule for a single machine to minimize the maximum lateness with release dates and precedence constraints. Our branch and bound algorithm uses four different heuristics to find upper bounds for the root branch node: heuristic ERD, modified Schrage's heuristic, heuristic BLOCK, and a variable neighborhood descent procedure. A lower bound is obtained by optimally solving the relaxed problem with preemption using Baker et al. [2]'s algorithm. At each subsequent branch node, two branches evolve from a schedule found by heuristic BLOCK using a binary branching rule using based on bottleneck job and critical job, for which a sufficient condition to recognize optimality is applied and a preemptive lower bound is compared with the current upper bound. To test the efficiency of the algorithm, we conduct extensive computational studies following the data generation framework proposed by Hall and Posner [9]. The branch and

bound algorithm solves 14,986 out of the 15,000 tested instances with up to 1000 jobs within 1 minute of CPU time using a desktop computer with 3 GHz CPU speed and 2 GB memory.

Acknowledgment

The author thanks the two reviewers for helpful comments on an earlier draft.

References

- [1] Baker KR. Introduction to sequencing and scheduling. New York: Wiley; 1974.
- [2] Baker KR, Lawler EL, Lenstra JK, Rinnooy Kan AHG. Preemptive scheduling of a single machine to minimize maximum cost subject to release dates and precedence constraints. *Operations Research* 1983;31:381–6.
- [3] Balas E, Vazacopoulos A. A guided local search with shifting bottleneck for job shop scheduling. *Management Science* 1998;44:262–75.
- [4] Brucker P. Scheduling algorithms, 2nd ed. Berlin: Springer; 1998.
- [5] Carlier J. The one-machine sequencing problem. *European Journal of Operational Research* 1982;11:42–7.
- [6] Garey MR, Johnson DS. Two-processor scheduling with starting time and deadlines. *SIAM Journal on Computing* 1977;6:416–26.
- [7] Graham RL, Lawler EL, Lenstra JK, Rinnooy Kan AHG. Optimization and approximation in deterministic machine scheduling: a survey. *Annals of Discrete Mathematics* 1979;5:287–326.
- [8] Hall NG, Liu Z, Potts CN. Rescheduling for multiple new orders. *INFORMS Journal on Computing* 2007;19:633–45.
- [9] Hall NG, Posner ME. Generating experimental data for computational testing with machine scheduling applications. *Operations Research* 2001;49:854–65.
- [10] Hall LA, Shmoys DB. Jackson's rule for single-machine scheduling: making a good heuristic better. *Mathematics of Operations Research* 1992;1:22–35.
- [11] Hansen P, Mladenović N. Variable neighborhood search: principles and applications. *European Journal of Operational Research* 2001;130:449–67.
- [12] Horn WA. Some simple scheduling algorithms. *Naval Research Logistics Quarterly* 1974;21:177–85.
- [13] Lageweg BJ, Lenstra JK, Rinnooy Kan AHG. Minimizing maximum lateness on one machine: computational experience and some applications. *Statistical Neerlandica* 1976;30:25–41.
- [14] Lawler EL. Optimal sequencing of a single machine subjects to precedence constraints. *Management Science* 1973;19:544–6.
- [15] Lenstra JK, Rinnooy Kan AHG, Brucker P. Complexity of machine scheduling problems. *Annals of Discrete Mathematics* 1977;1:343–62.
- [16] McMahon G, Florian M. On scheduling with ready times and due dates to minimize maximum lateness. *Operations Research* 1975;23:475–82.
- [17] Schrage LE. Obtaining optimal solution to resource constrained network scheduling problems. Unpublished manuscript, 1971.
- [18] Schutten JMJ, van de Velde SL, Zijm WH. Single machine scheduling with release dates, due dates and family setup times. *Management Science* 1996;42:1165–74.